

Brightness, Contrast_



Contents

1 0. 오리엔테이션

1.1 본 자료의 구성

1.2 화소 처리(point processing)와 변환 함수

2 1. 영상의 밝기 조절

2.1 밝기 조절이란

2.2 덧셈 연산과 포화(saturate) 연산

2.3 cv2.add() 함수

2.4 실습 코드 — 그레이스케일 밝기 조절

2.5 코드 들여다보기 — 사용한 함수·클래스 (1)

2.6 실행 결과 — dst3에서 확인하는 오버플로

2.7 실습 코드 — 컬러 영상 밝기 조절

2.8 코드 들여다보기 — 사용한 함수·클래스 (2)

2.9 실행 결과 — 컬러 밝기 조절

2.10 파이썬 파일(.py)로 정리한 형태

3 2. 영상의 기본 연산

3.1 덧셈 연산 — 두 영상 더하기

3.2 가중치 합(weighted sum)

3.3 평균 — 가중치 합의 특수한 경우

3.4 가중치를 바꿔 가며 본 합성 결과

3.5 cv2.addWeighted() 함수

[3.6 실습 코드 — 두 영상 블렌딩](#)

[3.7 코드 들여다보기 — 사용한 함수·클래스 \(3\)](#)

[3.8 뵤셈 연산](#)

[3.9 cv2.subtract\(\) 함수](#)

[3.10 실습 코드 — 뵤셈으로 배경 제거](#)

[3.11 코드 들여다보기 — 사용한 함수·클래스 \(4\)](#)

[3.12 차이 연산\(absdiff\)](#)

[3.13 cv2.absdiff\(\) 함수](#)

[3.14 실습 코드 — 차영상으로 움직임 찾기](#)

[3.15 코드 들여다보기 — 사용한 함수·클래스 \(5\)](#)

[4 3. 영상의 명암비 조절](#)

[4.1 명암비\(contrast\)란](#)

[4.2 곱셈으로 명암비 조절](#)

[4.3 효과적인 명암비 조절 — 128 기준](#)

[4.4 실습 코드 — 명암비 조절](#)

[4.5 코드 들여다보기 — 사용한 함수·클래스 \(6\)](#)

[4.6 실행 결과 — 명암비 조절](#)

[4.7 히스토그램 스트레칭](#)

[4.8 cv2.normalize\(\) 함수](#)

[4.9 실습 코드 — normalize로 스트레칭](#)

[4.10 코드 들여다보기 — 사용한 함수·클래스 \(7\)](#)

[4.11 실행 결과 — 히스토그램 스트레칭](#)

[4.12 히스토그램 평활화\(equalization\)](#)

[4.13 스트레칭 vs 평활화 비교](#)

4.14 실습 코드 — cv2.equalizeHist()

4.15 코드 들여다보기 — 사용한 함수·클래스 (8)

4.16 실행 결과 — 히스토그램 평활화

4.17 YCrCb 색공간

4.18 실습 코드 — 컬러 영상 히스토그램 평활화

4.19 코드 들여다보기 — 사용한 함수·클래스 (9)

4.20 실행 결과 — 컬러 평활화

5 **4. 색상 추출**

5.1 특정 범위의 행렬 원소 검출 — cv2.inRange()

5.2 왜 HSV를 쓰는가

5.3 실습 코드 — 특정 색상 추출

5.4 코드 들여다보기 — 사용한 함수·클래스 (10)

5.5 실행 결과 — BGR 기준 vs HSV 기준

6 **5. 정리**

6.1 함수 정리표

6.2 OpenCV 연산과 넘파이 연산의 차이

6.3 학습 포인트

Brightness, Contrast_

Exported from Confluence · <https://pinkwink.atlassian.net>

0. 오리엔테이션

본 자료의 구성

영상의 밝기 · 명암비 — 학습 흐름



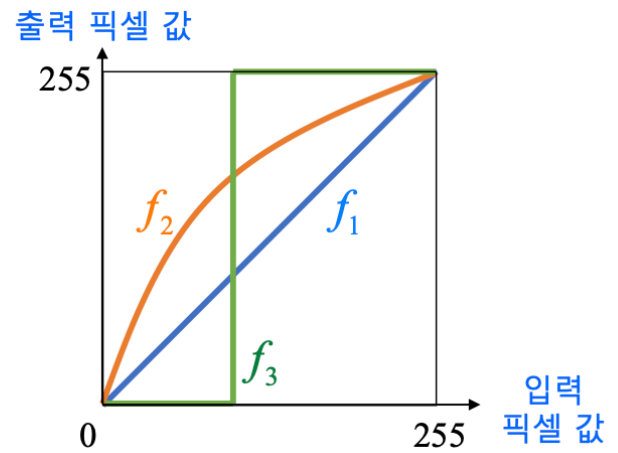
학습 순서

- 화소 처리의 기본 개념(변환 함수·포화 연산)에서 출발
- 밝기 조절(덧셈) → 두 영상의 기본 연산 → 명암비(곱셈) → 히스토그램 → 색상 추출 순으로 확장
- 각 주제마다 개념(그림) → 함수 시그니처 → 실습 코드 → 실행 결과 4단계 반복
- 실습 영상: lenna.bmp, sky.bmp, circle.jpg, road1-2.jpg, hawkes.png, field.bmp, candies.png

화소 처리(point processing)와 변환 함수

$$\text{dst}(x, y) = f(\text{src}(x, y))$$

변환 함수
(transfer function)

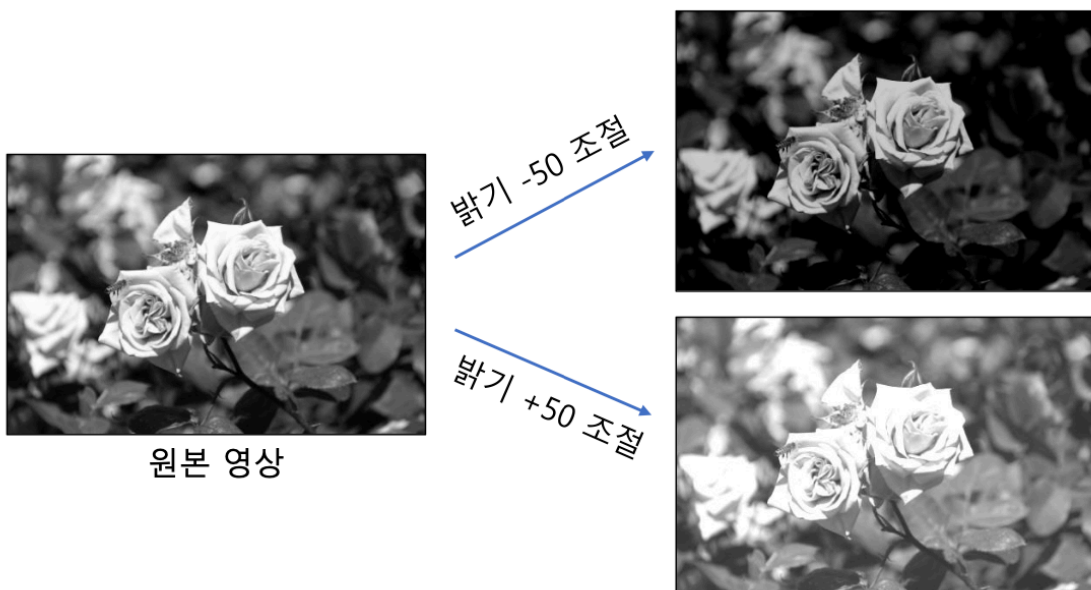


핵심 개념

- 화소 처리: 출력 화소값이 입력 화소값 하나로만 결정되는 연산 — $\text{dst}(x, y) = f(\text{src}(x, y))$
- 변환 함수(transfer function): 입력 화소값 → 출력 화소값 대응 규칙
- f_1 (직선, 기울기 1): 원본 그대로 / f_2 (위로 볼록): 전체적으로 밝아짐 / f_3 (계단): 특정 값 기준 이진화
- 이웃 화소를 참조하지 않으므로 연산이 단순하고 빠름(필터링과의 차이점)

1. 영상의 밝기 조절

밝기 조절이란

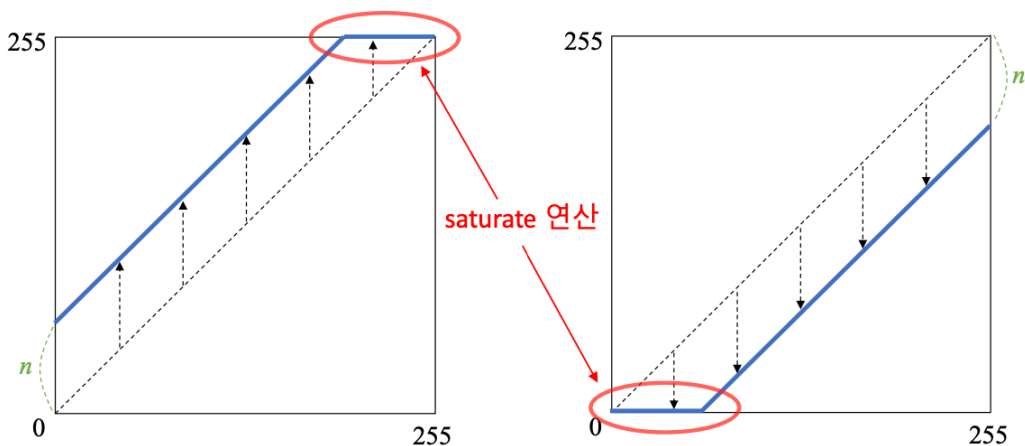


개념

- 모든 화소에 **같은 상수 n**을 더하거나 빼서 영상 전체를 밝게·어둡게 만드는 연산
- $n > 0$ 이면 밝아지고, $n < 0$ 이면 어두워짐
- 밝기만 이동할 뿐 화소 간 **상대적 차이(명암비)**는 그대로

덧셈 연산과 포화(saturate) 연산

$$\text{dst}(x, y) = \text{saturate}(\text{src}(x, y) + n)$$



핵심 개념

- $\text{dst}(x, y) = \text{saturate}(\text{src}(x, y) + n)$
- **saturate 연산**: 결과가 255를 넘으면 255로, 0보다 작으면 0으로 잘라 주는 처리
- 8비트 영상(`uint8`)의 표현 범위가 0~255이므로 반드시 필요
- 포화 처리를 하지 않으면 **값이 한 바퀴 돌아(wrap-around)** 밝은 곳이 갑자기 어두워지는 오류 발생

cv2.add() 함수

```
cv2.add(src1, src2, dst=None, mask=None, dtype=None) -> dst
```

- src1: (입력) 첫 번째 영상 또는 스칼라
- src2: (입력) 두 번째 영상 또는 스칼라
- dst: (출력) 덧셈 연산의 결과 영상
- mask: 마스크 영상
- dtype: 출력 영상(dst)의 타입. (e.g.) cv2.CV_8U, cv2.CV_32F 등
- 참고사항
 - 스칼라(Scalar)는 실수 값 하나 또는 실수 값 네 개로 구성된 튜플
 - dst를 함수 인자로 전달하려면 dst의 크기가 src1, src2와 같아야 하며, 타입이 적절해야 함

사용 포인트

- OpenCV의 `cv2.add()` 는 내부적으로 **saturate** 연산 포함
- 두 번째 인자에 **스칼라**를 주면 밝기 조절, **영상**을 주면 두 영상의 합성
- 컬러(3채널) 영상에 스칼라를 줄 때는 `(100, 100, 100, 0)` 처럼 **4원소 튜플** 형태로 전달

실습 코드 — 그레이스케일 밝기 조절

```
import cv2
import numpy as np
```

✓ 0.2s

Python

```
src = cv2.imread("./data/lenna.bmp", cv2.IMREAD_GRAYSCALE)

dst1 = cv2.add(src, 100)
dst2 = np.clip(src + 100.0, 0, 255).astype(np.uint8)
dst3 = src + 100

cv2.imshow("src", src)
cv2.imshow("dst1", dst1)
cv2.imshow("dst2", dst2)
cv2.imshow("dst3", dst3)
cv2.waitKey()

cv2.destroyAllWindows()
```

✓ 2.9s

Python

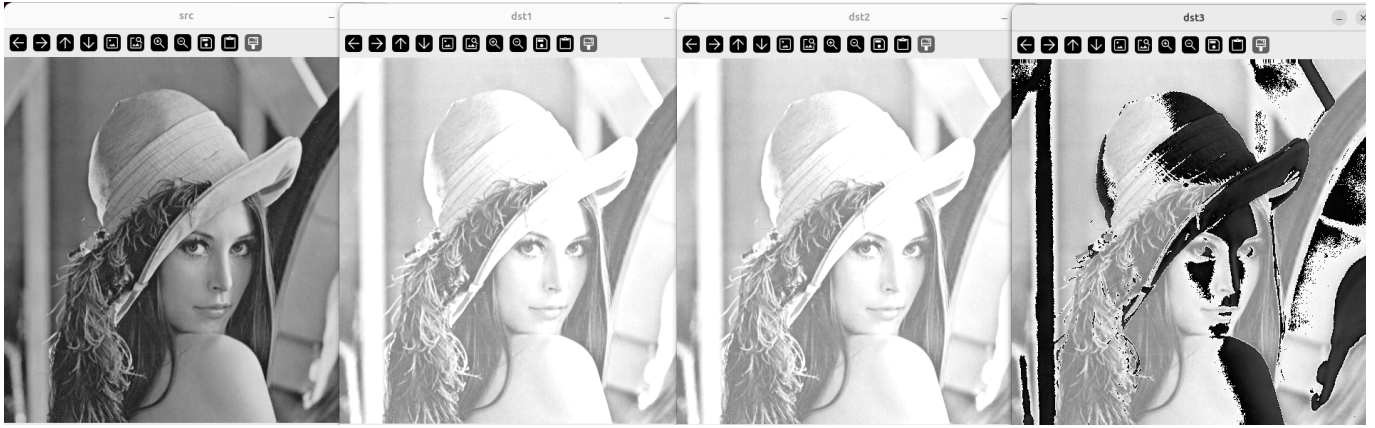
python

```
1 import cv2
2 import numpy as np
3
4 src = cv2.imread("./data/lenna.bmp", cv2.IMREAD_GRAYSCALE)
5
6 dst1 = cv2.add(src, 100) # saturate 포함
7 dst2 = np.clip(src + 100.0, 0, 255).astype(np.uint8) # 실수 변환 후 직접 클리핑
8 dst3 = src + 100 # uint8 오버플로 발생
9
10 cv2.imshow("src", src)
11 cv2.imshow("dst1", dst1)
12 cv2.imshow("dst2", dst2)
13 cv2.imshow("dst3", dst3)
14 cv2.waitKey()
15
16 cv2.destroyAllWindows()
```

코드 들여다보기 — 사용한 함수·클래스 (1)

이름	종류	설명
cv2	모듈	OpenCV 파이썬 바인딩
numpy (np)	모듈	행렬 연산 라이브러리
cv2.imread()	함수	영상 파일을 ndarray로 읽음. 실패 시 None 반환
cv2.IMREAD_GRAYSCALE	상수	1채널 그레이스케일로 읽으라는 플래그
cv2.add()	함수	덧셈 + saturate → 255 초과분을 255로 고정
np.clip()	함수	지정 범위(0~255) 밖의 값을 잘라 냄
.astype()	메서드	실수 계산 결과를 <code>np.uint8</code> 로 되돌림
src + 100	연산	넘파이 uint8 순수 덧셈 → 포화 없이 wrap-around (비교용 잘못된 예)
cv2.imshow()	함수	결과 영상을 창에 출력
cv2.waitKey()	함수	키 입력 대기(인자 없으면 무한 대기)
cv2.destroyAllWindows()	함수	열린 모든 창 닫기

실행 결과 — dst3에서 확인하는 오버플로



- `dst1` · `dst2` : 전체가 자연스럽게 밝아짐(포화 처리 정상)
- `dst3` : 원래 밝던 부분이 검게 뒤집힘 → 255를 넘어 0부터 다시 세는 wrap-around
- 결론: 영상 연산에서는 `cv2.add()` 또는 `np.clip()`을 반드시 사용

실습 코드 — 컬러 영상 밝기 조절

```
src = cv2.imread("./data/lenna.bmp")

dst1 = cv2.add(src, (100, 100, 100, 0))
dst2 = np.clip(src + 100.0, 0, 255).astype(np.uint8)

cv2.imshow("src", src)
cv2.imshow("dst1", dst1)
cv2.imshow("dst2 ", dst2)
cv2.waitKey()

cv2.destroyAllWindows()
```

✓ 1.0s

Python

python

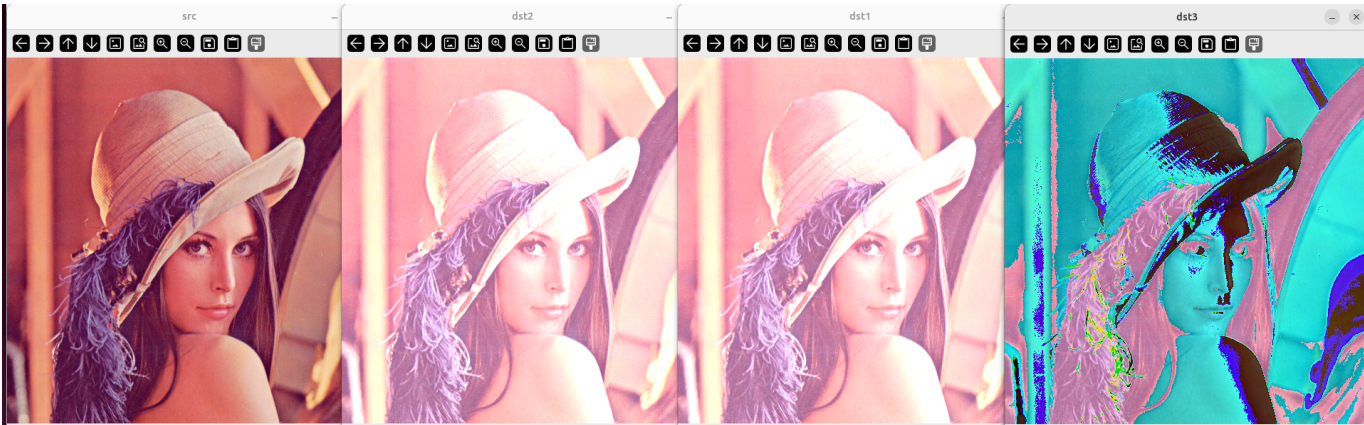
```
1  src = cv2.imread("./data/lenna.bmp")          # 컬러(BGR 3채널)로 읽기
2
3  dst1 = cv2.add(src, (100, 100, 100, 0))        # 각 채널에 +100 (4원소 스칼라)
4  dst2 = np.clip(src + 100.0, 0, 255).astype(np.uint8)
5
6  cv2.imshow("src", src)
7  cv2.imshow("dst1", dst1)
8  cv2.imshow("dst2", dst2)
9  cv2.waitKey()
10
11 cv2.destroyAllWindows()
```

코드 들여다보기 — 사용한 함수·클래스 (2)

이름	종류	설명
<code>cv2.imread()</code>	함수	플래그 없이 호출 → BGR 3채널 컬러로 읽음
<code>cv2.add(src, (100,100,100,0))</code>	함수	B·G·R 각 채널에 100을 더함. 네 번째 원소는 알파 자리(사용 안 함)
<code>(100, 100, 100, 0)</code>	스칼라	OpenCV 스칼라는 최대 4원소 튜플 → 3채널 영상엔 앞 3개만 적용
<code>np.clip(src + 100.0, 0, 255)</code>	연산	실수 승격 후 범위 제한 — <code>cv2.add()</code> 와 같은 효과

컬러 영상에 `cv2.add(src, 100)` 처럼 단일 숫자를 주면 첫 번째 채널(B)에만 더해져 색이 파랗게 들어짐. 반드시 튜플로 전달할 것.

실행 결과 — 컬러 밝기 조절



- 세 채널을 동일하게 올리면 색감 유지한 채 전체가 밝아짐
- 포화 처리를 생략한 경우(`dst3`) 채널별로 값이 뒤집혀 색이 완전히 어긋남

파이썬 파일(.py)로 정리한 형태

openCV > 08_bright_contrast_01.py > ...

```
1  import sys
2  import numpy as np
3  import cv2
4
5  src = cv2.imread("./lenna.bmp", cv2.IMREAD_GRAYSCALE)
6
7  if src is None:
8      print("Image load failed!")
9      sys.exit()
10
11  dst1 = cv2.add(src, 100)
12  dst2 = np.clip(src + 100.0, 0, 255).astype(np.uint8)
13
14  cv2.imshow("src", src)
15  cv2.imshow("dst1", dst1)
16  cv2.imshow("dst2", dst2)
17  cv2.waitKey()
18
19  src = cv2.imread("lenna.bmp")
20
21  if src is None:
22      print("Image load failed!")
23      sys.exit()
24
25  dst1 = cv2.add(src, (100, 100, 100, 0))
26  dst2 = np.clip(src + 100.0, 0, 255).astype(np.uint8)
27
28  cv2.imshow("src", src)
29  cv2.imshow("dst1", dst1)
30  cv2.imshow("dst2 ", dst2)
31  cv2.waitKey()
32
33  cv2.destroyAllWindows()
```



스크립트 작성 포인트

- 노트북과 달리 `if src is None:` 로드 실패 검사를 넣고 `sys.exit()` 로 종료
- 그레이스케일 예제와 컬러 예제를 한 파일에 순서대로 배치
- 각 블록 끝에 `cv2.waitKey()` 를 뒤 결과를 확인한 뒤 다음 단계로 진행

2. 영상의 기본 연산

덧셈 연산 — 두 영상 더하기


```
cv2.add(src1, src2, dst=None, mask=None, dtype=None) -> dst
```

- src1: (입력) 첫 번째 영상 또는 스칼라
- src2: (입력) 두 번째 영상 또는 스칼라
- dst: (출력) 덧셈 연산의 결과 영상
- mask: 마스크 영상
- dtype: 출력 영상(dst)의 타입. (e.g.) cv2.CV_8U, cv2.CV_32F 등
- 참고사항
 - 스칼라(Scalar)는 실수 값 하나 또는 실수 값 네 개로 구성된 튜플
 - dst를 함수 인자로 전달하려면 dst의 크기가 src1, src2와 같아야 하며, 타입이 적절해야 함

정리

- $dst(x, y) = \text{saturate}(src1(x, y) + src2(x, y))$
- 두 영상은 크기와 타입이 같아야 함 → 다르면 `cv2.resize()` 로 맞춤
- 그냥 더하면 대부분 영역이 255로 포화되어 하얗게 됨 → 실무에선 **가중치 합**을 사용

가중치 합(weighted sum)

$$dst(x, y) = \text{saturate}(\alpha \cdot src1(x, y) + \beta \cdot src2(x, y))$$

- 두 영상의 같은 위치에 존재하는 픽셀 값에 대하여 가중합을 계산하여 결과 영상의 픽셀 값으로 설정
- 보통 $\alpha + \beta = 1$ 이 되도록 설정 → 두 입력 영상의 평균 밝기를 유지

핵심 개념

- $dst(x, y) = \text{saturate}(\alpha \cdot src1(x, y) + \beta \cdot src2(x, y))$
- 보통 $\alpha + \beta = 1$ 로 설정 → 두 영상의 평균 밝기 유지
- α 가 클수록 첫 번째 영상이 진하게 보임 → **영상 블렌딩(합성)**의 기본

평균 — 가중치 합의 특수한 경우

- 가중치를 $\alpha = \beta = 0.5$ 로 설정한 가중치 합

$$\text{dst}(x, y) = \frac{1}{2}(\text{src1}(x, y) + \text{src2}(x, y))$$

- $\alpha = \beta = 0.5$ 이면 두 영상의 산술 평균
- 감시 영상 여러 장을 평균하면 잡음이 줄어드는 효과도 있음

가중치를 바꿔 가며 본 합성 결과

$$\frac{1}{2} \left(\text{img1} + \text{img2} \right) = \text{result}$$



$\alpha = 1, \beta = 0$



$\alpha = 0.75, \beta = 0.25$



$\alpha = 0.5, \beta = 0.5$



$\alpha = 0.25, \beta = 0.75$



$\alpha = 0, \beta = 1$

- $\alpha = 1$ 이면 첫 영상만, $\alpha = 0$ 이면 두 번째 영상만 보임
- 중간값에서는 두 영상이 겹쳐 보이는 **디졸브(dissolve)** 효과
- 영상 전환·워터마크·이중 노출 표현에 활용

cv2.addWeighted() 함수

```
cv2.addWeighted(src1, alpha, src2, beta, gamma, dst=None, dtype=None) -> dst
```

- src1: 첫 번째 영상
- alpha: 첫 번째 영상 가중치
- src2: 두 번째 영상. src1과 같은 크기 & 같은 타입
- beta: 두 번째 영상 가중치
- gamma: 결과 영상에 추가적으로 더할 값
- dst: 가중치 합 결과 영상
- dtype: 출력 영상(dst)의 타입

사용 포인트

- `cv2.addWeighted(src1, alpha, src2, beta, gamma)`
- `gamma` 는 결과에 추가로 더하는 밝기 보정값(보통 0)
- 두 입력 영상의 크기·타입이 반드시 동일해야 함

실습 코드 — 두 영상 블렌딩

```
import matplotlib.pyplot as plt
```

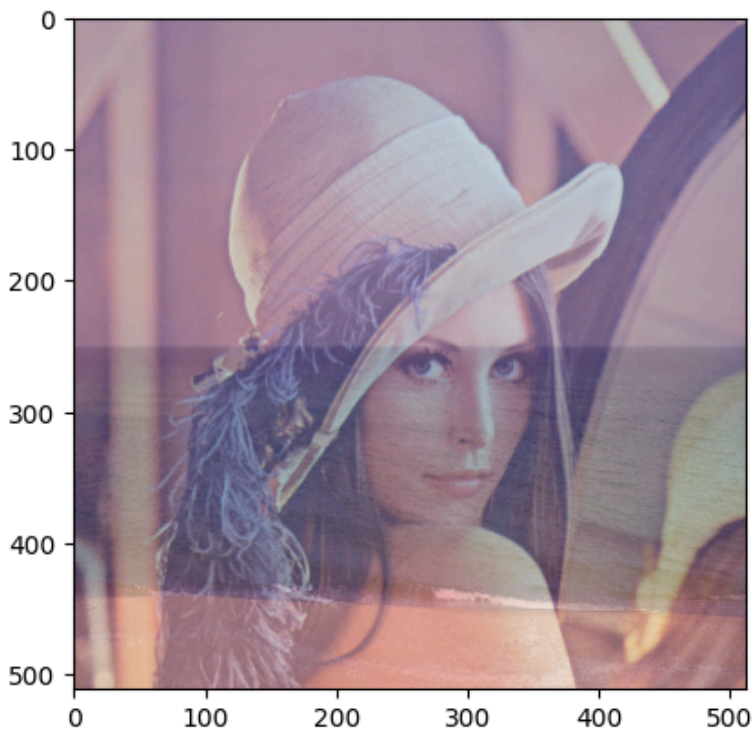
✓ 0.6s

Python

```
src1 = cv2.imread('./data/lenna.bmp')
src2 = cv2.imread('./data/sky.bmp')
resize_src2 = cv2.resize(src2, (src1.shape[1], src1.shape[0]))
dst = cv2.addWeighted(src1=src1, alpha=0.5, src2=resize_src2, beta=0.5, gamma=0)
show_dst = cv2.cvtColor(dst, cv2.COLOR_BGR2RGB)
plt.imshow(show_dst);
```

✓ 0.2s

Python



python

```
1 import matplotlib.pyplot as plt
2
3 src1 = cv2.imread('./data/lenna.bmp')
4 src2 = cv2.imread('./data/sky.bmp')
5
6 resize_src2 = cv2.resize(src2, (src1.shape[1], src1.shape[0])) # 크기 맞추기
7 dst = cv2.addWeighted(src1=src1, alpha=0.5, src2=resize_src2, beta=0.5, gamma=0)
8
9 show_dst = cv2.cvtColor(dst, cv2.COLOR_BGR2RGB) # matplotlib용 색 순
   서 변환
10 plt.imshow(show_dst);
```

코드 들여다보기 — 사용한 함수·클래스 (3)

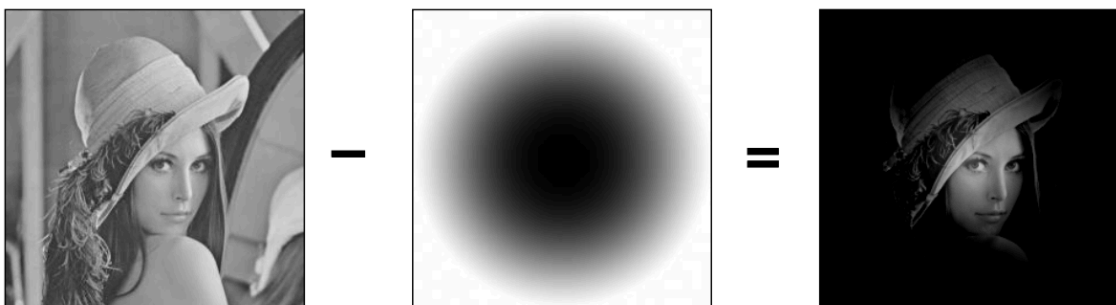
이름	종류	설명
matplotlib.pyplot (plt)	모듈	노트북에서 영상을 인라인으로 표시
cv2.imread()	함수	두 장의 컬러 영상을 각각 읽음
src1.shape	속성	(높이, 너비, 채널) 튜플 → <code>shape[1]</code> =너비, <code>shape[0]</code> =높이
cv2.resize()	함수	src2를 src1과 같은 크기로 변환(크기가 달라 발생하는 오류 예방)
cv2.addWeighted()	함수	$\alpha \cdot \text{src1} + \beta \cdot \text{src2} + \gamma$ 계산 — $\alpha=\beta=0.5$ 로 반반 합성
cv2.cvtColor()	함수	OpenCV의 BGR → matplotlib의 RGB 순서로 변환
cv2.COLOR_BGR2RGB	상수	채널 순서 변환 플래그
plt.imshow()	함수	노트북 셀에 결과 영상 표시

OpenCV는 **BGR**, matplotlib은 **RGB** 순서를 사용함. `cvtColor` 를 빠뜨리면 빨강·파랑이 뒤바뀐 색으로 보임.

뺄셈 연산

$$\text{dst}(x, y) = \text{saturate}(\text{src1}(x, y) - \text{src2}(x, y))$$

- 두 영상의 같은 위치에 존재하는 픽셀 값에 대하여 뺄셈 연산을 수행하여 결과 영상의 픽셀 값으로 설정
- 뺄셈 결과가 0보다 작으면 픽셀 값을 0으로 설정 (포화 연산)



핵심 개념

- $\text{dst}(x, y) = \text{saturate}(\text{src1}(x, y) - \text{src2}(x, y))$
- 결과가 0보다 작으면 0으로 고정(포화 연산)
- 순서에 따라 결과가 달라짐 — $\text{src1} - \text{src2} \neq \text{src2} - \text{src1}$
- 특정 패턴(마스크·조명 성분)을 빼내 관심 영역만 남기는 용도로 사용

cv2.subtract() 함수

```
cv2.subtract(src1, src2, dst=None, mask=None, dtype=None) -> dst
```

- src1: 첫 번째 영상 또는 스칼라
- src2: 두 번째 영상 또는 스칼라
- dst: 뺄셈 연산 결과 영상
- mask: 마스크 영상
- dtype: 출력 영상(dst)의 타입

사용 포인트

- 포화 처리 포함 — 음수는 0으로 잘림
- 스칼라를 빼면 영상 전체가 어두워짐(밝기 조절의 반대 방향)

실습 코드 — 뺄셈으로 배경 제거

```
src1 = cv2.imread('./data/lenna.bmp', cv2.IMREAD_GRAYSCALE)
src2 = cv2.imread('./data/circle.jpg', cv2.IMREAD_GRAYSCALE)

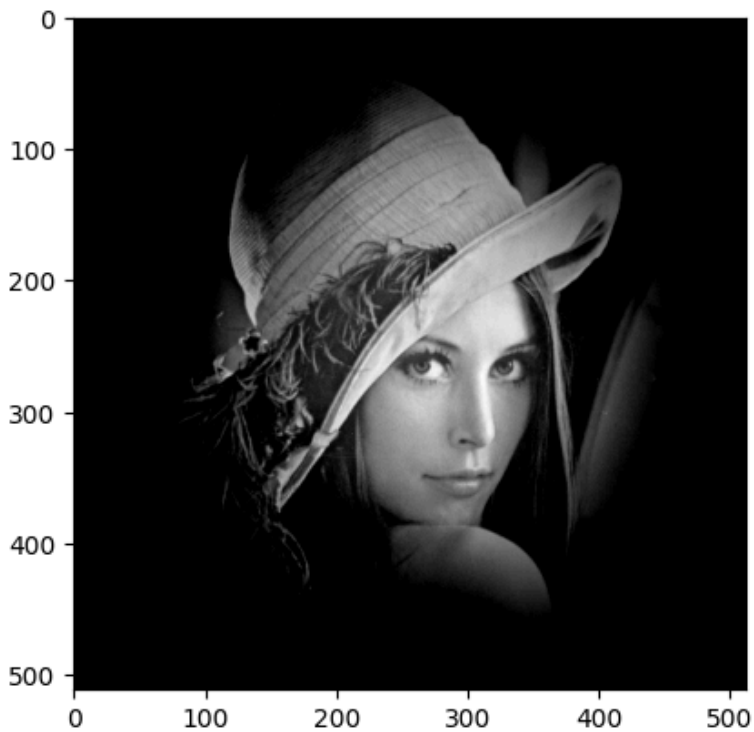
src2 = cv2.resize(src2, (src1.shape[1], src1.shape[0]))

dst = cv2.subtract(src1, src2)

plt.imshow(dst, cmap='gray');
```

✓ 0.2s

Python



python

```
1 src1 = cv2.imread('./data/lenna.bmp', cv2.IMREAD_GRAYSCALE)
2 src2 = cv2.imread('./data/circle.jpg', cv2.IMREAD_GRAYSCALE)
3
4 src2 = cv2.resize(src2, (src1.shape[1], src1.shape[0]))
5
6 dst = cv2.subtract(src1, src2)
7
8 plt.imshow(dst, cmap='gray');
```

코드 들여다보기 — 사용한 함수·클래스 (4)

이름	종류	설명
cv2.imread(..., IMREAD_GRAYSCALE)	함수	두 영상을 모두 1채널로 읽어 타입을 일치시킴

cv2.resize()	함수	circle.jpg를 lenna.bmp와 같은 크기로 맞춤
cv2.subtract()	함수	src1 - src2를 포화 연산으로 계산
plt.imshow(dst, cmap='gray')	함수	1채널 영상을 회색조 컬러맵으로 표시(생략 시 색이 입혀져 보임)

- 중앙이 검은 원형 그라데이션을 빼면 가장자리는 0으로 잘리고 가운데만 남음
- 비네팅(주변부 어둡게) 같은 연출에도 응용 가능

차이 연산(absdiff)

$$\text{dst}(x, y) = |\text{src1}(x, y) - \text{src2}(x, y)|$$

- 두 영상의 같은 위치에 존재하는 픽셀 값에 대하여 뺄셈 연산을 수행한 후, 그 절댓값을 결과 영상의 픽셀 값으로 설정
- 뺄셈 연산과 달리 입력 영상의 순서에 영향을 받지 않음



핵심 개념

- $\text{dst}(x, y) = |\text{src1}(x, y) - \text{src2}(x, y)|$ — 뺄셈 후 절댓값
- 뺄셈과 달리 입력 순서에 무관
- 같은 위치에서 촬영한 두 장면의 차이를 구하면 움직인 물체만 밝게 남음
- 고정 카메라 기반 움직임 검출·침입 감지의 기초

cv2.absdiff() 함수


```
cv2.absdiff(src1, src2, dst=None) -> dst
```

- src1: 첫 번째 영상 또는 스칼라
- src2: 두 번째 영상 또는 스칼라
- dst: 차이 연산 결과 영상(차영상)

사용 포인트

- 인자는 두 영상(또는 스칼라)뿐 — 마스크·dtype 옵션 없음
- 결과를 임계값 처리(`cv2.threshold`)하면 움직임 영역 마스크를 얻을 수 있음

실습 코드 — 차영상으로 움직임 찾기

```
img1 = cv2.imread('./data/road1.jpg', cv2.IMREAD_GRAYSCALE)
img2 = cv2.imread('./data/road2.jpg', cv2.IMREAD_GRAYSCALE)

dst = cv2.absdiff(img1, img2)

fig = plt.figure(figsize=(10,7))

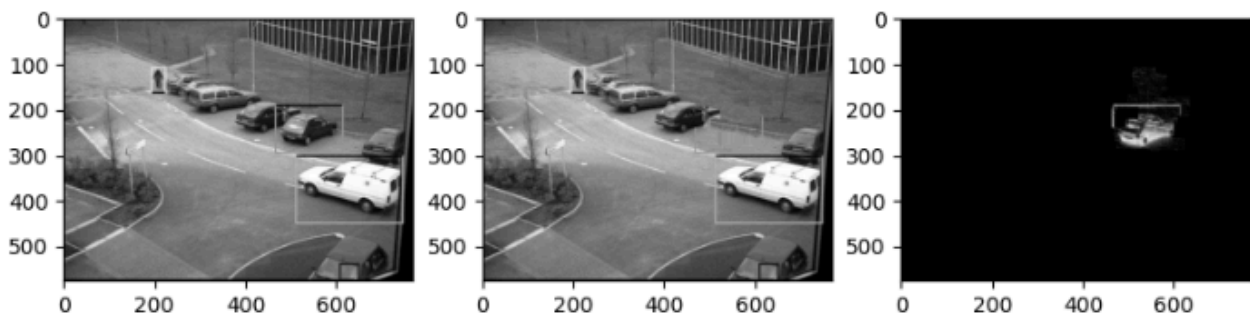
fig.add_subplot(1,3,1)
plt.imshow(img1, cmap='gray')

fig.add_subplot(1,3,2)
plt.imshow(img2, cmap='gray')

fig.add_subplot(1,3,3)
plt.imshow(dst, cmap='gray');
```

✓ 0.5s

Python



python

```
1  img1 = cv2.imread('./data/road1.jpg', cv2.IMREAD_GRAYSCALE)
2  img2 = cv2.imread('./data/road2.jpg', cv2.IMREAD_GRAYSCALE)
3
4  dst = cv2.absdiff(img1, img2)
5
6  fig = plt.figure(figsize=(10, 7))
7
8  fig.add_subplot(1, 3, 1)
9  plt.imshow(img1, cmap='gray')
10
11 fig.add_subplot(1, 3, 2)
12 plt.imshow(img2, cmap='gray')
13
14 fig.add_subplot(1, 3, 3)
15 plt.imshow(dst, cmap='gray');
```

코드 들여다보기 — 사용한 함수·클래스 (5)

이름	종류	설명
cv2.absdiff()	함수	두 프레임의 화소별 절대 차이 계산
plt.figure(figsize=(10,7))	함수	그림 전체 크기(인치) 지정
fig.add_subplot(1, 3, n)	메서드	1행 3열 배치에서 n번째 칸 선택
plt.imshow(..., cmap='gray')	함수	각 칸에 영상 표시
Figure	클래스	matplotlib의 그림 객체 — 여러 subplot을 담는 컨테이너

- 배경(도로·건물)은 두 영상이 같아 차이가 0 → 검게 표시
- 이동한 차량 위치만 밝게 남음 → **단순 뿔셈만으로 움직임 검출**
- 실제 CCTV에서는 조명 변화·그림자 때문에 배경 모델링 기법을 추가로 사용

3. 영상의 명암비 조절

명암비(contrast)란



명암비가 낮은 영상



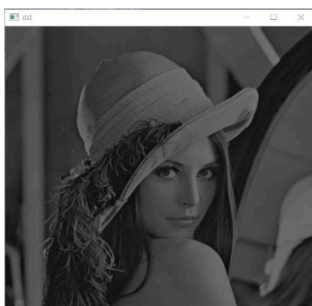
명암비가 높은 영상

핵심 개념

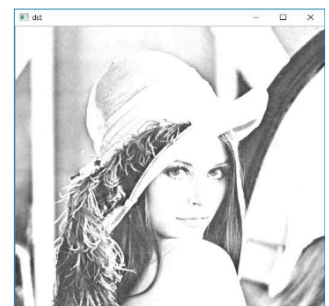
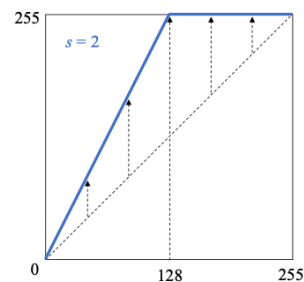
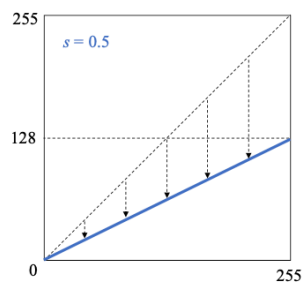
- **명암비**: 영상에서 밝은 곳과 어두운 곳의 **차이 정도**
- 명암비가 낮으면 전체가 뿌옇게(회색에 몰림), 높으면 또렷하게 보임
- 밝기 조절은 **더하기**, 명암비 조절은 **곱하기**가 기본

곱셈으로 명암비 조절

$$\text{dst}(x, y) = \text{saturnate}(s \cdot \text{src}(x, y))$$



$s = 0.5$



$s = 2.0$

정리

- $\text{dst}(x, y) = \text{saturate}(s \cdot \text{src}(x, y))$
- $s > 1$ 이면 명암비 증가(밝은 곳이 더 밝아짐), $s < 1$ 이면 감소
- 단순 곱셈은 전체 밝기까지 함께 변함($s > 1$ 이면 영상이 통째로 밝아지고 쉽게 포화)

효과적인 명암비 조절 — 128 기준

개선 아이디어

- 중간 밝기 128을 기준으로 잡고 기준에서 떨어진 만큼만 확대
- $\text{dst} = \text{src} + (\text{src} - 128) \cdot \alpha = (1 + \alpha) \cdot \text{src} - 128 \cdot \alpha$
- $\alpha > 0$ 이면 명암비 증가, $-1 < \alpha < 0$ 이면 감소
- 128 부근의 평균 밝기는 유지되므로 전체가 밝아지거나 어두워지지 않음

실습 코드 — 명암비 조절

```
src = cv2.imread('./data/lenna.bmp', cv2.IMREAD_GRAYSCALE)

alpha = 1.0
dst = np.clip((1+alpha)*src - 128*alpha, 0, 255).astype(np.uint8)

cv2.imshow('src', src)
cv2.imshow('dst', dst)
cv2.waitKey()
cv2.destroyAllWindows()
```

7.6s

Python

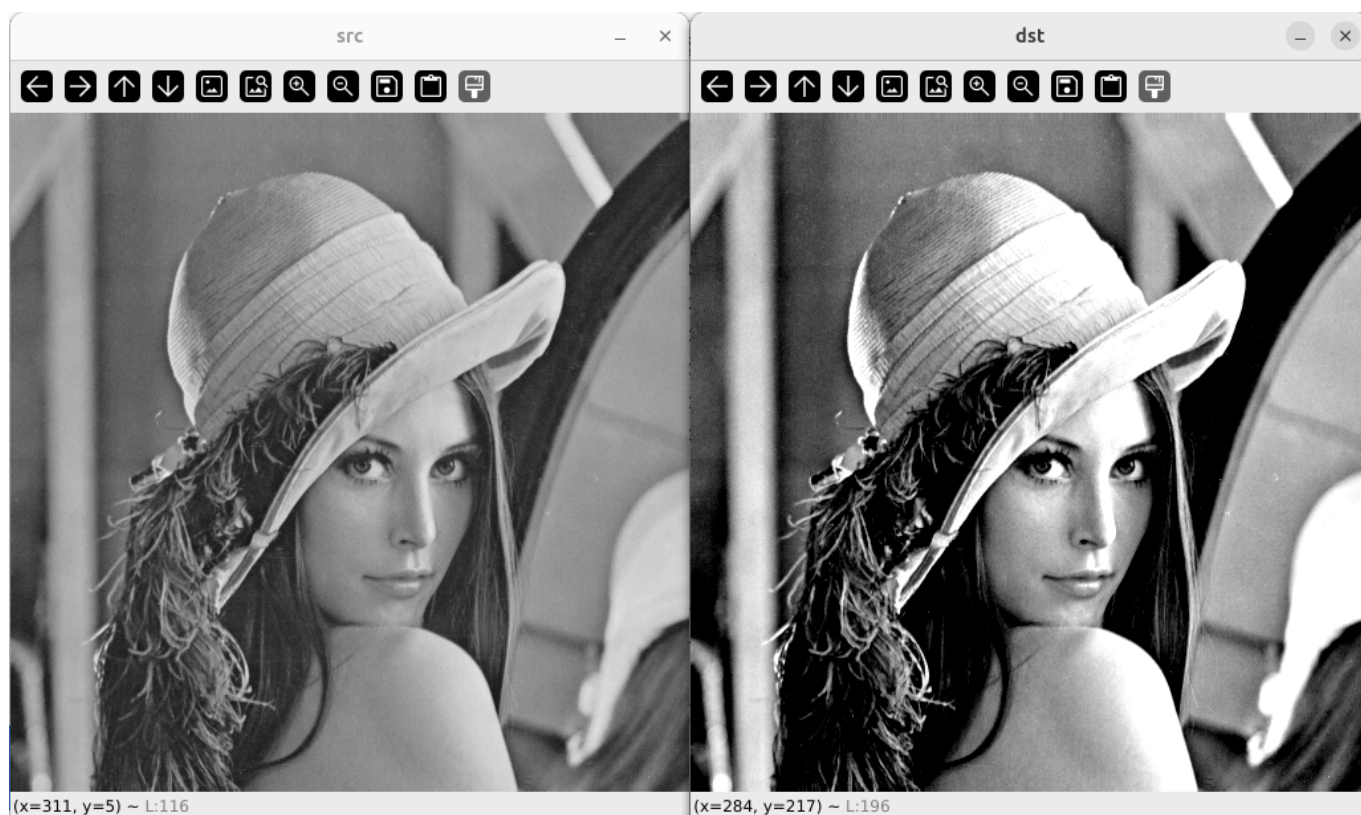
python

```
1  src = cv2.imread('./data/lenna.bmp', cv2.IMREAD_GRAYSCALE)
2
3  alpha = 1.0
4  dst = np.clip((1+alpha)*src - 128*alpha, 0, 255).astype(np.uint8)
5
6  cv2.imshow('src', src)
7  cv2.imshow('dst', dst)
8  cv2.waitKey()
9  cv2.destroyAllWindows()
```

코드 들여다보기 — 사용한 함수·클래스 (6)

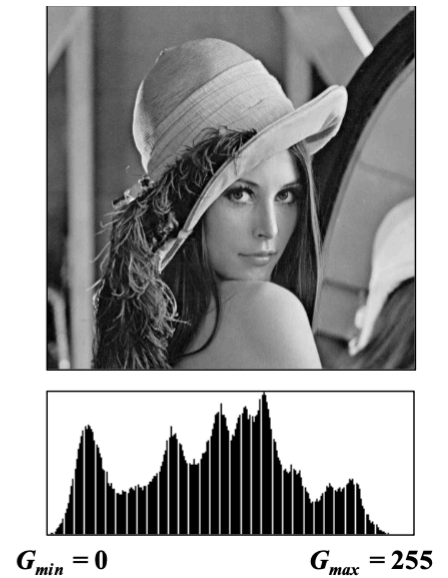
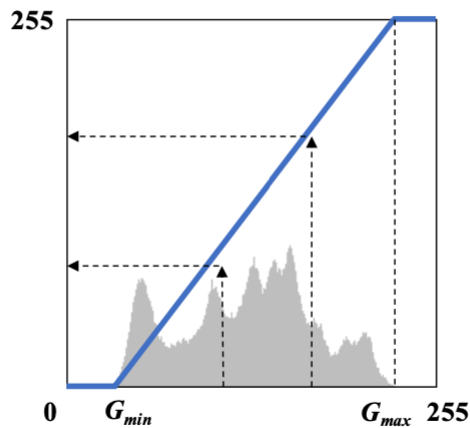
이름	종류	설명
alpha	변수	명암비 조절 강도. 1.0이면 기울기 2배의 강한 대비
$(1+\alpha)*src - 128*\alpha$	연산	128을 기준으로 화소값을 확대·축소하는 변환 함수
np.clip()	함수	0~255 범위로 포화 처리
.astype(np.uint8)	메서드	영상 자료형으로 복원
cv2.imshow() / waitKey() / destroyAllWindows()	함수	원본·결과 비교 출력, 키 대기, 창 정리

실행 결과 — 명암비 조절



- 밝은 부분은 더 밝게, 어두운 부분은 더 어둡게 → 윤곽이 또렷해짐
- α 를 너무 키우면 밝은 영역이 255에, 어두운 영역이 0에 몰려 계조(디테일)가 사라짐

히스토그램 스트레칭



핵심 개념

- **히스토그램**: 화소값(0~255)별 개수 분포 그래프
- 명암비가 낮은 영상 = 히스토그램이 **좁은 구간**에 몰려 있음
- **스트레칭**: 실제 사용 중인 구간 $[G_{min}, G_{max}]$ 를 **0~255 전체**로 늘려 퍼는 변환
- 수식: $dst = (src - G_{min}) \cdot 255 / (G_{max} - G_{min})$
- 분포 모양은 그대로 두고 가로로 늘리는 선형 변환

cv2.normalize() 함수

```
cv2.normalize(src, dst, alpha=None, beta=None, norm_type=None, dtype=None, mask=None) -> dst
```

- **src**: 입력 영상
- **dst**: 결과 영상
- **alpha**: (노름 정규화인 경우) 목표 노름 값,
(원소 값 범위 정규화인 경우) 최솟값
- **beta**: (원소 값 범위 정규화인 경우) 최댓값
- **norm_type**: 정규화 타입. NORM_INF, NORM_L1, NORM_L2, NORM_MINMAX.
- **dtype**: 결과 영상의 타입
- **mask**: 마스크 영상

사용 포인트

- `norm_type=cv2.NORM_MINMAX` + `alpha=0, beta=255` → 히스토그램 스트레칭과 동일
- `alpha`·`beta`의 의미는 `norm_type`에 따라 달라짐(원소 범위 정규화일 때 최솟값·최댓값)
- `NORM_INF`·`NORM_L1`·`NORM_L2`는 벡터 노름 기준 정규화(영상보다는 특징 벡터에 사용)

실습 코드 — normalize로 스트레칭

openCV > 08_bright_contrast_02_normalize.py > ...

```
1  import sys
2  import numpy as np
3  import cv2
4
5  src = cv2.imread("./data/hawkes.png", cv2.IMREAD_GRAYSCALE)
6  src = cv2.resize(src, (567, 379))
7
8  if src is None:
9      print("Image load failed!")
10     sys.exit()
11
12  dst = cv2.normalize(src, None, 0, 255, cv2.NORM_MINMAX)
13
14  cv2.imshow("src", src)
15  cv2.imshow("dst", dst)
16  cv2.waitKey()
17
18  cv2.destroyAllWindows()
```

python

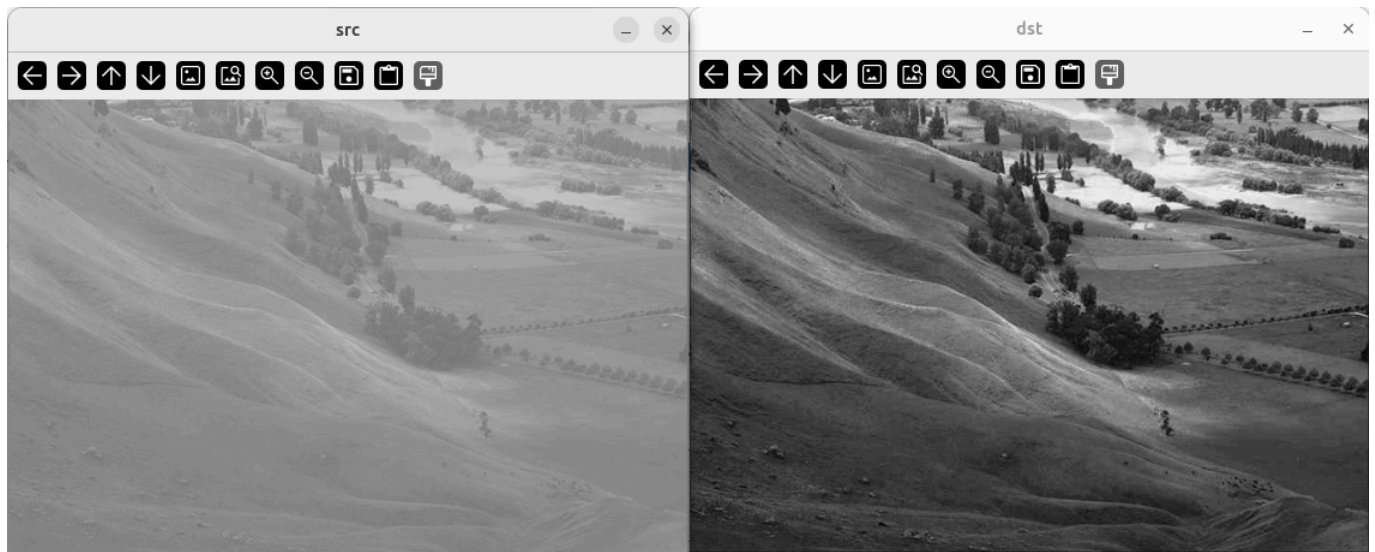
```
1 import sys
2 import numpy as np
3 import cv2
4
5 src = cv2.imread("./data/hawkes.png", cv2.IMREAD_GRAYSCALE)
6 src = cv2.resize(src, (567, 379))
7
8 if src is None:
9     print("Image load failed!")
10    sys.exit()
11
12 dst = cv2.normalize(src, None, 0, 255, cv2.NORM_MINMAX)
13
14 cv2.imshow("src", src)
15 cv2.imshow("dst", dst)
16 cv2.waitKey()
17
18 cv2.destroyAllWindows()
```

코드 들여다보기 — 사용한 함수·클래스 (7)

이름	종류	설명
cv2.resize(src, (567, 379))	함수	화면에 맞게 영상 크기 축소
cv2.normalize()	함수	화소값 범위를 재조정. 두 번째 인자 <code>None</code> 은 출력 배열을 새로 만들라는 뜻
0, 255	인자	NORM_MINMAX에서 결과의 최솟값·최댓값
cv2.NORM_MINMAX	상수	최소·최대 기준 선형 정규화 = 히스토그램 스트레칭
sys.exit()	함수	영상 로드 실패 시 종료

위 코드는 `cv2.resize()` 가 `if src is None` 검사보다 앞에 있음. 파일이 없으면 검사 전에 `resize`에서 오류가 남 → 실제 스크립트에서는 **로드 검사 다음에 `resize`를 두는 것이 안전함**.

실행 결과 — 히스토그램 스트레칭

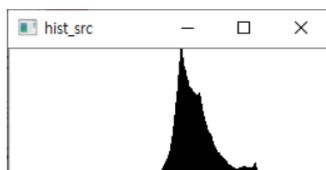


- 뿌영던 원본이 어두운 곳~밝은 곳까지 고르게 퍼져 **선명해짐**
- 밝기 분포가 이미 0~255에 걸쳐 있는 영상은 스트레칭 효과가 거의 없음

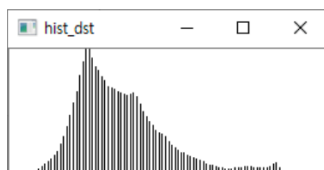
히스토그램 평활화(equalization)

스트레칭과의 차이

- **스트레칭**: 분포를 **선형**으로 늘림 — 모양은 유지
- **평활화**: 누적분포함수(CDF)를 이용해 **분포 자체를 고르게** 재배치 — 비선형
- 화소가 몰린 구간은 넓게, 드문 구간은 좁게 매핑 → 결과 히스토그램이 평탄해짐
- 대비 개선 효과가 크지만 **잡음까지 강조**될 수 있음



입력 영상



히스토그램 스트레칭



히스토그램 평활화

- 가운데(스트레칭)는 자연스러운 대비 개선, 오른쪽(평활화)은 훨씬 강한 대비
- 평활화 결과 히스토그램은 빈 구간이 생기며 **막대가 성기게 퍼짐**

스트레칭 vs 평활화 비교

항목	히스토그램 스트레칭	히스토그램 평활화
함수	<code>cv2.normalize(..., NORM_MINMAX)</code>	<code>cv2.equalizeHist()</code>
변환 성격	선형(1차식)	비선형(누적분포 기반)
분포 변화	모양 유지, 좌우로 늘림	고르게 재배치
강도	온건함	강함
적합한 경우	일부 구간에만 몰린 영상	전체적으로 대비가 약한 영상
주의점	이미 넓게 퍼진 영상엔 효과 없음	잡음·과도한 대비 발생 가능

실습 코드 — cv2.equalizeHist()

openCV > 08_bright_contrast_03_histogram_equalize.py > ...

```
1  import sys
2  import numpy as np
3  import cv2
4
5  src = cv2.imread("./data/hawkes.png", cv2.IMREAD_GRAYSCALE)
6  src = cv2.resize(src, (567, 379))
7
8  if src is None:
9      print("Image load failed!")
10     sys.exit()
11
12  dst = cv2.equalizeHist(src)
13
14  cv2.imshow("src", src)
15  cv2.imshow("dst", dst)
16  cv2.waitKey()
17
18  cv2.destroyAllWindows()
19
20  # 컬러 영상의 히스토그램 평활화
21  src = cv2.imread("field.bmp")
22
23  if src is None:
24      print("Image load failed!")
25      sys.exit()
26
27  src_ycrb = cv2.cvtColor(src, cv2.COLOR_BGR2YCrCb)
28  ycrb_planes = cv2.split(src_ycrb)
29
30  # 밝기 성분에 대해서만 히스토그램 평활화 수행
31  ycrb_planes[0] = cv2.equalizeHist(ycrb_planes[0])
32
33  dst_ycrb = cv2.merge(ycrb_planes)
34  dst = cv2.cvtColor(dst_ycrb, cv2.COLOR_YCrCb2BGR)
35
36  cv2.imshow("src", src)
37  cv2.imshow("dst", dst)
38  cv2.waitKey()
39
40  cv2.destroyAllWindows()
```

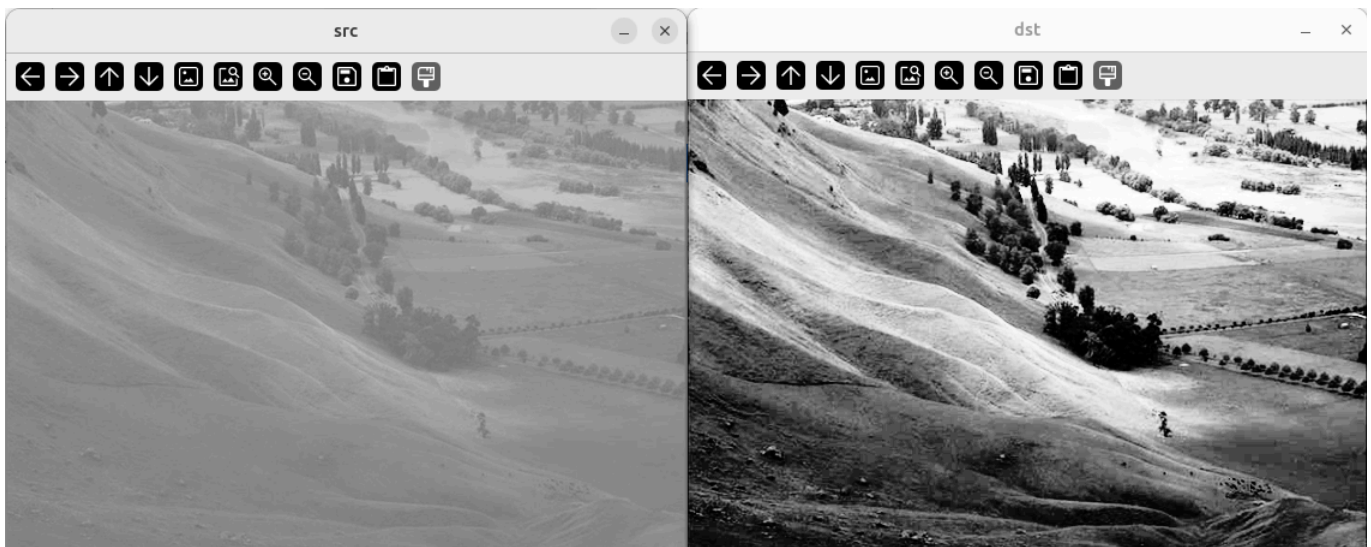
python

```
1 import sys
2 import numpy as np
3 import cv2
4
5 src = cv2.imread("./data/hawkes.png", cv2.IMREAD_GRAYSCALE)
6 src = cv2.resize(src, (567, 379))
7
8 if src is None:
9     print("Image load failed!")
10    sys.exit()
11
12 dst = cv2.equalizeHist(src)
13
14 cv2.imshow("src", src)
15 cv2.imshow("dst", dst)
16 cv2.waitKey()
17
18 cv2.destroyAllWindows()
```

코드 들여다보기 — 사용한 함수·클래스 (8)

이름	종류	설명
cv2.equalizeHist()	함수	히스토그램 평활화 수행. 1채널(그레이스케일) 영상만 입력 가능
cv2.imread(..., IMREAD_GRAYSCALE)	함수	평활화 입력 조건에 맞춰 1채널로 읽음
cv2.resize()	함수	표시용 크기 조정

실행 결과 — 히스토그램 평활화



- 스트레칭보다 대비가 훨씬 강하게 살아남
- 하늘·능선의 미세한 질감까지 드러나지만 **노이즈도 함께 두드러짐**

YCrCb 색공간

개념 정의

- **YCbCr(YCrCb)**: 영상 시스템에서 사용되는 색공간의 일종
- **Y** = 휘도(밝기) 성분, **Cb·Cr** = 색차 성분
- 밝기와 색을 분리하므로 **밝기만 손대는 처리에 유리**
- 컬러 영상 평활화를 BGR 채널별로 하면 **색이 심하게 왜곡됨** → Y 채널만 평활화

실습 코드 — 컬러 영상 히스토그램 평활화

```
import sys
import numpy as np
import cv2

src = cv2.imread('./data/field.bmp')

if src is None :
    print('Image load failed!')
    sys.exit()

src_ycrb = cv2.cvtColor(src, cv2.COLOR_BGR2YCrCb)
y, cr, cb = cv2.split(src_ycrb)

y = cv2.equalizeHist(y)

dst_ycrb = cv2.merge([y, cr, cb])
dst = cv2.cvtColor(dst_ycrb, cv2.COLOR_YCrCb2BGR)

cv2.imshow('src', src)
cv2.imshow('dst', dst)

cv2.waitKey()
cv2.destroyAllWindows()
```

✓ 8.7s

Python

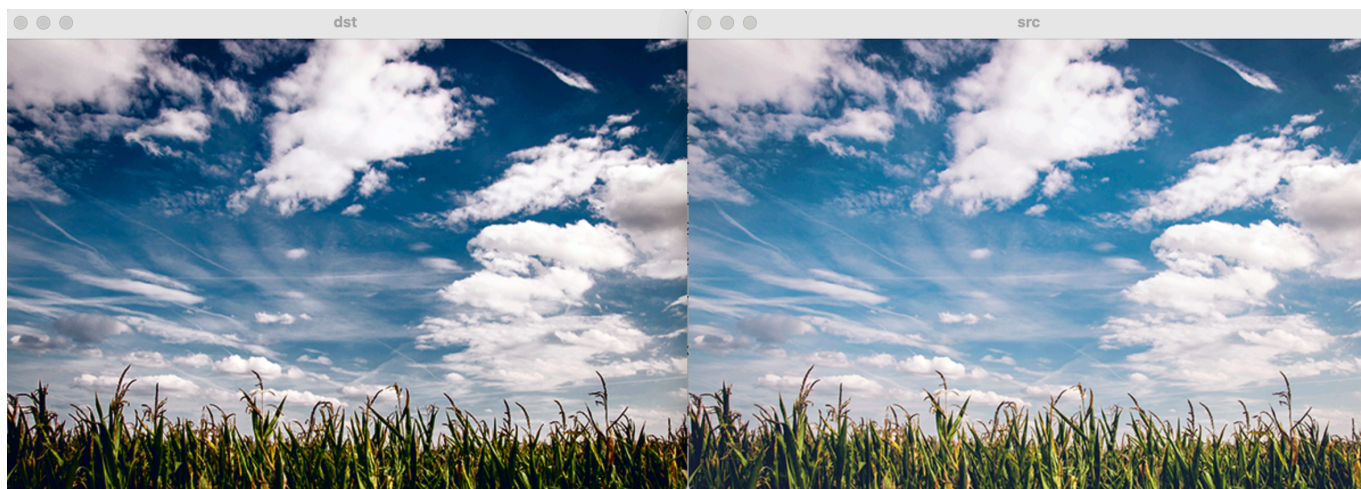
python

```
1  import sys
2  import numpy as np
3  import cv2
4
5  src = cv2.imread('./data/field.bmp')
6
7  if src is None:
8      print('Image load failed!')
9      sys.exit()
10
11  src_ycrcb = cv2.cvtColor(src, cv2.COLOR_BGR2YCrCb)
12  y, cr, cb = cv2.split(src_ycrcb)
13
14  y = cv2.equalizeHist(y)                # 밝기 성분만 평활화
15
16  dst_ycrcb = cv2.merge([y, cr, cb])
17  dst = cv2.cvtColor(dst_ycrcb, cv2.COLOR_YCrCb2BGR)
18
19  cv2.imshow('src', src)
20  cv2.imshow('dst', dst)
21
22  cv2.waitKey()
23  cv2.destroyAllWindows()
```

코드 들여다보기 — 사용한 함수·클래스 (9)

이름	종류	설명
cv2.cvtColor()	함수	색 공간 변환 수행
cv2.COLOR_BGR2YCrCb	상수	BGR → YCrCb(밝기·색차 분리) 변환 플래그
cv2.split()	함수	다채널 영상을 채널별 1채널 영상 리스트로 분리
cv2.equalizeHist(y)	함수	밝기 채널만 평활화 → 색상 왜곡 방지
cv2.merge()	함수	분리했던 채널들을 다시 하나의 다채널 영상으로 합침
cv2.COLOR_YCrCb2BGR	상수	처리 후 다시 BGR로 되돌리는 플래그

실행 결과 — 컬러 평활화



- 하늘의 파랑이 진해지고 구름·수풀의 명암이 뚜렷해짐
- 색상(Cr·Cb)은 건드리지 않았으므로 색감은 유지

4. 색상 추출

특정 범위의 행렬 원소 검출 — cv2.inRange()

```
cv2.inRange(src, lowerb, upperb, dst=None) -> dst
```

- src: 입력 행렬
- lowerb: 하한 값 행렬 또는 스칼라
- upperb: 상한 값 행렬 또는 스칼라
- dst: 입력 영상과 같은 크기의 마스크 영상. (numpy.uint8)
범위 안에 들어가는 픽셀은 255, 나머지는 0으로 설정.
 - 단일 채널: $\text{dst}(I) = \text{lowerb}(I)_0 \leq \text{src}(I)_0 \leq \text{upperb}(I)_0$
 - 다중 채널: $\text{dst}(I) = \text{lowerb}(I)_0 \leq \text{src}(I)_0 \leq \text{upperb}(I)_0 \wedge \text{lowerb}(I)_1 \leq \text{src}(I)_1 \leq \text{upperb}(I)_1 \wedge$

사용 포인트

- `lowerb` ≤ 화소값 ≤ `upperb` 인 위치만 **255**, 나머지는 **0**인 마스크 영상 생성
- 다중 채널이면 **모든 채널이 동시에** 범위 조건을 만족해야 255
- 결과는 `numpy.uint8` 1채널 마스크 → 이후 `bitwise_and` 등에 활용

왜 HSV를 쓰는가

핵심 개념

- **BGR**: 밝기와 색상이 뒤섞여 있어 조명이 바뀌면 범위 지정이 어려움
- **HSV**: **H**(색상)·**S**(채도)·**V**(명도)로 분리 → 색상만 H 범위로 지정 가능
- OpenCV의 H 범위는 **0~179**(360°를 절반으로 압축), S·V는 0~255
- 같은 색이라도 조명이 달라지면 V가 변하므로 **V·S는 넉넉히, H는 좁게** 잡는 것이 요령

실습 코드 — 특정 색상 추출

openCV > 08_bright_contrast_04_color_extraxt.py > ...

```
1  import sys
2  import numpy as np
3  import cv2
4
5
6  src1 = cv2.imread("./data/candies.png")
7
8  if src1 is None:
9      print("Image 1 load failed!")
10     sys.exit()
11
12  src2 = cv2.imread("./data/candies2.png")
13
14  if src2 is None:
15      print("Image 2 load failed!")
16      sys.exit()
17
18  src1_hsv = cv2.cvtColor(src1, cv2.COLOR_BGR2HSV)
19  src2_hsv = cv2.cvtColor(src2, cv2.COLOR_BGR2HSV)
20
21  dst1_1 = cv2.inRange(src1, (0, 128, 0), (100, 255, 100))
22  dst1_2 = cv2.inRange(src1_hsv, (50, 150, 0), (80, 255, 255))
23  dst2_1 = cv2.inRange(src2, (0, 128, 0), (100, 255, 100))
24  dst2_2 = cv2.inRange(src2_hsv, (50, 150, 0), (80, 255, 255))
25
26  cv2.imshow("src1", src1)
27  cv2.imshow("dst1-1", dst1_1)
28  cv2.imshow("dst1-2", dst1_2)
29  cv2.imshow("src2", src2)
30  cv2.imshow("dst2-1", dst2_1)
31  cv2.imshow("dst2-2", dst2_2)
32  cv2.waitKey()
33
34  cv2.destroyAllWindows()
```

python

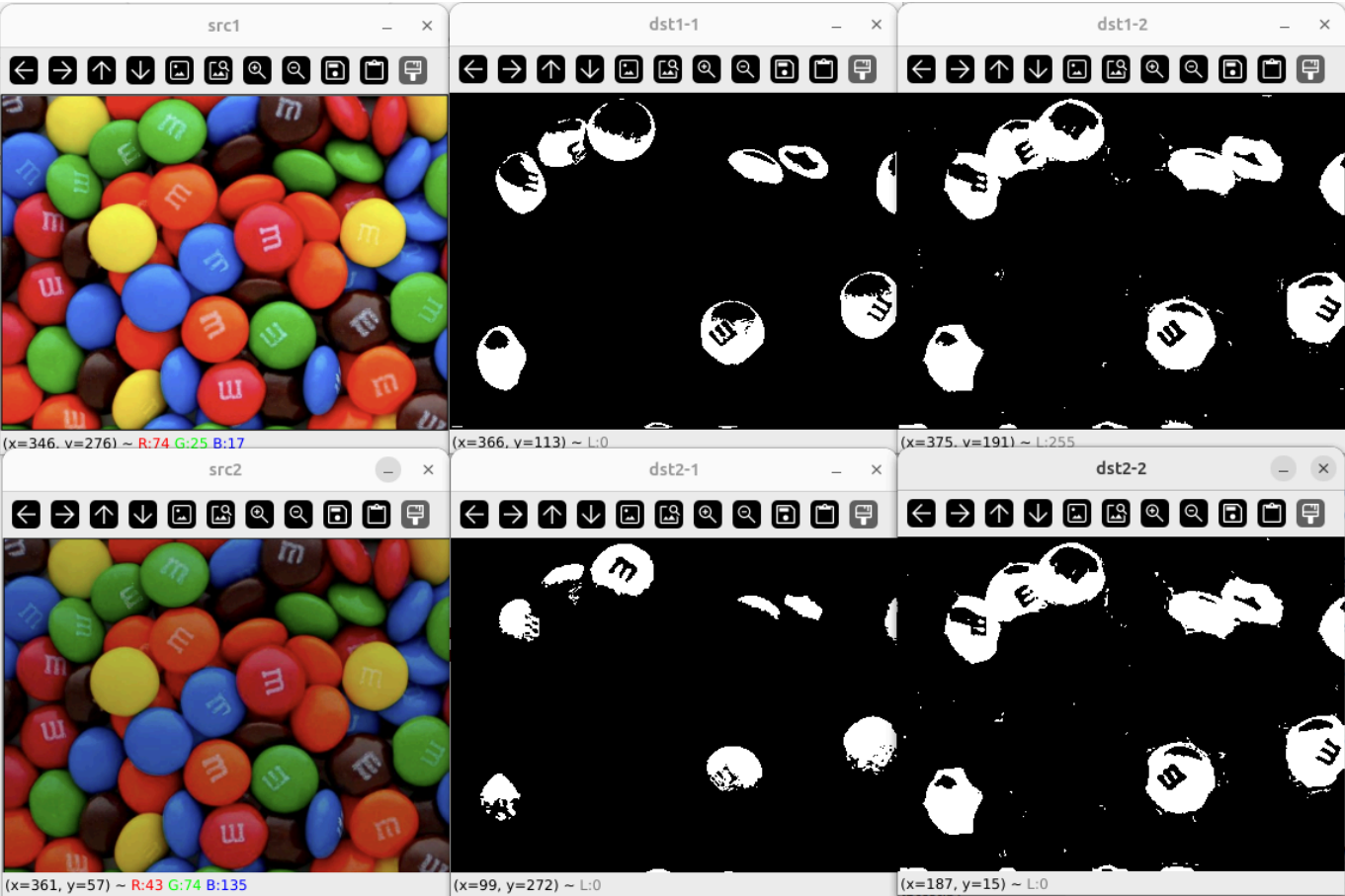
```
1  import sys
2  import numpy as np
3  import cv2
4
5  src1 = cv2.imread("./data/candies.png")
6
7  if src1 is None:
8      print("Image 1 load failed!")
9      sys.exit()
10
11  src2 = cv2.imread("./data/candies2.png")
12
13  if src2 is None:
14      print("Image 2 load failed!")
15      sys.exit()
16
17  src1_hsv = cv2.cvtColor(src1, cv2.COLOR_BGR2HSV)
18  src2_hsv = cv2.cvtColor(src2, cv2.COLOR_BGR2HSV)
19
20  dst1_1 = cv2.inRange(src1, (0, 128, 0), (100, 255, 100)) # BGR 기준 초록
21  dst1_2 = cv2.inRange(src1_hsv, (50, 150, 0), (80, 255, 255)) # HSV 기준 초록
22  dst2_1 = cv2.inRange(src2, (0, 128, 0), (100, 255, 100))
23  dst2_2 = cv2.inRange(src2_hsv, (50, 150, 0), (80, 255, 255))
24
25  cv2.imshow("src1", src1)
26  cv2.imshow("dst1-1", dst1_1)
27  cv2.imshow("dst1-2", dst1_2)
28  cv2.imshow("src2", src2)
29  cv2.imshow("dst2-1", dst2_1)
30  cv2.imshow("dst2-2", dst2_2)
31  cv2.waitKey()
32
33  cv2.destroyAllWindows()
```

코드 들여다보기 — 사용한 함수·클래스 (10)

이름	종류	설명
cv2.imread()	함수	밝기 조건이 다른 두 장의 사탕 사진을 읽음
cv2.cvtColor()	함수	BGR → HSV 색공간 변환
cv2.COLOR_BGR2HSV	상수	HSV 변환 플래그(H: 0~179, S·V: 0~255)
cv2.inRange()	함수	지정 범위에 드는 화소만 255인 마스크 생성
(0, 128, 0) ~ (100, 255, 100)	인자	BGR 기준 초록 범위 — B·R은 낮고 G는 높은 화소

(50, 150, 0) ~ (80, 255, 255)	인자	HSV 기준 초록 범위 — H 50~80(초록 계열), S 150 이상(선명한 색)
cv2.imshow()	함수	원본과 두 방식의 마스크를 나란히 비교 출력

실행 결과 — BGR 기준 vs HSV 기준



- dst1-1 · dst2-1 (BGR 기준): 조명이 다른 두 사진에서 검출 결과가 크게 달라짐
- dst1-2 · dst2-2 (HSV 기준): 두 사진 모두에서 초록 사탕이 안정적으로 검출됨
- 결론: 색상 기반 검출은 HSV(또는 YCrCb)에서 수행하는 것이 기본

5. 정리

함수 정리표

함수	수식·역할	포화 처리	주 용도
----	-------	-------	------

<code>cv2.add()</code>	$\text{src1} + \text{src2}$ (또는 스칼라)	O	밝기 올리기, 영상 합
<code>cv2.subtract()</code>	$\text{src1} - \text{src2}$	O	밝기 내리기, 배경 제거
<code>cv2.absdiff()</code>	$ \text{src1} - \text{src2} $	불필요(항상 0 이상)	움직임 검출·차영상
<code>cv2.addWeighted()</code>	$\alpha \cdot \text{src1} + \beta \cdot \text{src2} + \gamma$	O	영상 블렌딩·디졸브
<code>cv2.normalize()</code>	범위 [alpha, beta]로 선형 사상	O	히스토그램 스트레칭
<code>cv2.equalizeHist()</code>	누적분포 기반 재배치	O	히스토그램 평활화(1채널)
<code>cv2.inRange()</code>	범위 내 255 / 외 0	해당 없음	색상 추출 마스크
<code>cv2.cvtColor()</code>	색 공간 변환	해당 없음	YCrCb·HSV 전환

OpenCV 연산과 넘파이 연산의 차이

구분	<code>cv2.add()</code> 등 OpenCV 연산	넘파이 순수 연산(<code>src + 100</code>)
범위 초과 처리	saturate — 255/0으로 고정	wrap-around — 값이 한 바퀴 뛴
결과	의도한 밝기 변화	밝은 부분이 검게 뒤집힘
안전한 사용법	그대로 사용	<code>np.clip(src + 100.0, 0, 255).astype(np.uint8)</code>
컬러 스칼라	(100, 100, 100, 0) 튜플 필요	브로드캐스팅으로 전 채널 적용

학습 포인트

- 화소 처리 = **변환 함수 f** 하나로 표현 — 밝기는 +n, 명암비는 $\times s$
- 8비트 영상에서는 **포화(saturate)** 처리가 필수 — 넘파이 순수 연산은 오버플로 주의
- 명암비는 **128을 기준으로** 조절해야 전체 밝기가 흔들리지 않음
- 히스토그램 **스트레칭(선형)**과 **평활화(비선형)**의 차이를 구분할 것
- 컬러 영상의 밝기·대비 처리는 **YCrCb의 Y 채널**만 — BGR 채널별 처리는 색 왜곡
- 색상 검출은 **HSV**에서 — 조명 변화에 강함



PinkLAB YouTube Channel

PinkLAB Studio — Robotics, AI, ROS2 and more!

 **Subscribe on YouTube**

https://www.youtube.com/@pinklab_studio